P_7

October 3, 2026

0.1 Practical 7 - Polymorphism and Encapsulation

Exercise 1: Magic Methods

1. Create a class called Point
2. Add an **init** method with x and y, and store them as properties
3. Add a **repr** method that returns f"Point({x}, {y})"
4. Create an object p1 with x 2 and y 3
5. Print p1

[3]:
```
class Point:

    def __init__(self, x, y):
        self.x = x
        self.y = y

    def __repr__(self):
        return f"Point({self.x}, {self.y})"

p1 = Point(2,3)
print(p1)
```

```
Point(2, 3)
```

Exercise 2: Inheritance

1. Create a parent class Animal with an **init** that takes name
2. Add a method speak that prints the name
3. Create a child class Dog that inherits from Animal
4. Create an object d1 = Dog("Rex")
5. call d1.speak()

[11]:
```
class Animal:

    def __init__(self, name):
        self.name = name

    def speak(self):
        print(self.name)
```

```
class Dog(Animal):
    pass

d1 = Dog("Rex")
d1.speak()
```

```
Rex
```

Exercise 3: Polymorphism

1. Create a class Cat with a method sound that prints “Meow”
2. Create a class Fox with a method sound that prints “Wa-pa-pa-pa-pa-pow!”
3. Create objects c1 = Cat() and f1 = Fox()
4. Call sound() on both objects

[7]:
```
class Cat:

    def sound(self):
        print("Meow")

class Fox:

    def sound(self):
        print("Wa-pa-pa-pa-pa-pow!")

c1 = Cat()
f1 = Fox()

for animal in (c1,f1):
    animal.sound()
```

```
Meow
Wa-pa-pa-pa-pa-pow!
```

Exerceise 4: Encapsulation

1. Create a class ScoreBoard
2. Add an **init** with a score parameter and store it as a private attribute
3. Add a method called get_score that returns the private score
4. Create an object s1 with a score of 0
5. Print the score of s1

[9]:
```
class ScoreBoard:

    def __init__(self, score):
        self.__score = score

    def get_score(self):
        return self.__score
```

```
s1 = ScoreBoard(0)
print(s1.get_score())
```

```
0
```